Analysis and Design Report for VStay

Villa Booking System

Student: Nguyen Anh Khoi
Student ID: SE194126
Class: SE1936
Course: SWD392
Lecturer: Nguyen Dang Hoang Tuan

May 27, 2026

Contents

1 Introduction

VStay is a villa booking system developed for a software analysis and design assignment. The system allows guests to browse villas, search and filter villa information, view villa details, create bookings, cancel bookings, and make simulated payments. It also provides administration features for managing villas and booking statuses.

The problem comes from small accommodation businesses that often manage villa information, bookings, and payments manually through phone calls, messages, or notes. Without a centralized system, information can become inconsistent and booking records can be difficult to track. VStay focuses on the essential workflows required for an academic software design project.

1.1 Project Summary

Item	Description
Project name	VStay Villa Booking System
Course	SWD392 - Software Design
Main users	Guest and Admin
Main purpose	Analyze, design, and implement a simple villa booking backend prototype for an academic assignment.
Core features	Villa browsing, booking management, payment simulation, villa administration, and booking status management.
Technology stack	Java, Spring Boot, Maven, Spring Data JPA, Spring Security, Swagger/OpenAPI, and H2 demo database.
Repository	https://github.com/kenji-cmyk/mini-booking-villa

1.2 Objectives

- Allow guests to view, search, filter, and inspect villa details.
- Allow guests to create, view, cancel bookings, and make payments.
- Allow admins to create, update, delete, and view villa records.
- Allow admins to view all bookings and update booking statuses.
- Provide complete analysis and design artifacts, including requirements, UML diagrams, design patterns, and an implementation prototype.

1.3 Scope

The project scope is intentionally limited to core villa booking features, villa management, booking management, and simplified payment handling. Advanced commercial features such as AI recommendations, loyalty programs, chat, multi-vendor marketplace support, mobile applications, dynamic pricing, and advanced financial reports are out of scope.

2 System Requirements

2.1 Actors

Actor	Description
Guest	A user who browses villas, searches and filters villas, creates bookings, cancels bookings, makes payments, and checks booking or payment status.
Admin	A system user responsible for managing villa information and updating booking status.

2.2 Functional Requirements

ID	Requirement Summary
FR-01–FR-04	Guests can view the villa list, search, filter, and view villa details.
FR-05–FR-09	Guests can create bookings, while the system validates data, checks availability, shows booking details, and allows eligible cancellation.
FR-10–FR-12	Guests can submit a simulated payment, the system records payment information, and guests can view payment status.
FR-13–FR-16	Admins can create, update, delete, and view villa records. Villas with active bookings cannot be deleted.
FR-17–FR-20	Admins can view all bookings, view booking details, and update booking status according to valid business rules.

2.3 Non-Functional Requirements

- **Performance:** villa lists, search results, and booking details should respond within about 3 seconds for demo data.
- **Security:** admin functions must be restricted to authorized admin users; sensitive payment data must not be fully exposed.
- **Reliability:** the system must prevent duplicate active bookings for the same villa and overlapping dates.
- **Maintainability:** the backend must clearly separate controller, service, repository, entity, DTO, mapper, payment, and exception responsibilities.
- **Validation:** required fields, booking dates, guest count, and payment information must be validated at system boundaries.

3 Use Case Diagram

3.1 AI Prompt Used

```
Create a UML use case diagram for a villa booking management system.

Requirements:
- Guests can view villas, search villas, view villa details, create bookings,
cancel bookings, make payments, and view payment status.
- Admins can create, update, delete, and manage villas and bookings.
```

```
Generate a PlantUML use case diagram using Guest and Admin actors.
Keep the diagram simple and suitable for a university software engineering
assignment.
```

3.2 UML Code

```
@startuml
left to right direction

actor Guest
actor Admin

rectangle "VStay Villa Booking System" {

  usecase "UC-01\nView Villas" as UC01
  usecase "UC-02\nSearch and Filter Villas" as UC02
  usecase "UC-03\nView Villa Details" as UC03

  usecase "UC-04\nCreate Booking" as UC04
  usecase "Validate Booking Information" as UC04A
  usecase "Check Villa Availability" as UC04B

  usecase "UC-05\nView Booking Details" as UC05
  usecase "UC-06\nCancel Booking" as UC06
  usecase "UC-07\nMake Payment" as UC07
  usecase "UC-08\nView Payment Status" as UC08

  usecase "UC-09\nCreate Villa" as UC09
  usecase "UC-10\nUpdate Villa" as UC10
  usecase "UC-11\nDelete Villa" as UC11
  usecase "UC-12\nView Villa List" as UC12
  usecase "UC-13\nView All Bookings" as UC13
  usecase "UC-14\nUpdate Booking Status" as UC14
}

Guest --> UC01
Guest --> UC02
Guest --> UC03
Guest --> UC04
Guest --> UC05
Guest --> UC06
Guest --> UC07
Guest --> UC08

Admin --> UC09
Admin --> UC10
Admin --> UC11
Admin --> UC12
Admin --> UC13
Admin --> UC14
```

```
UC04 ..> UC04A : <<include>>
UC04 ..> UC04B : <<include>>

@enduml
```

3.3 UML Image

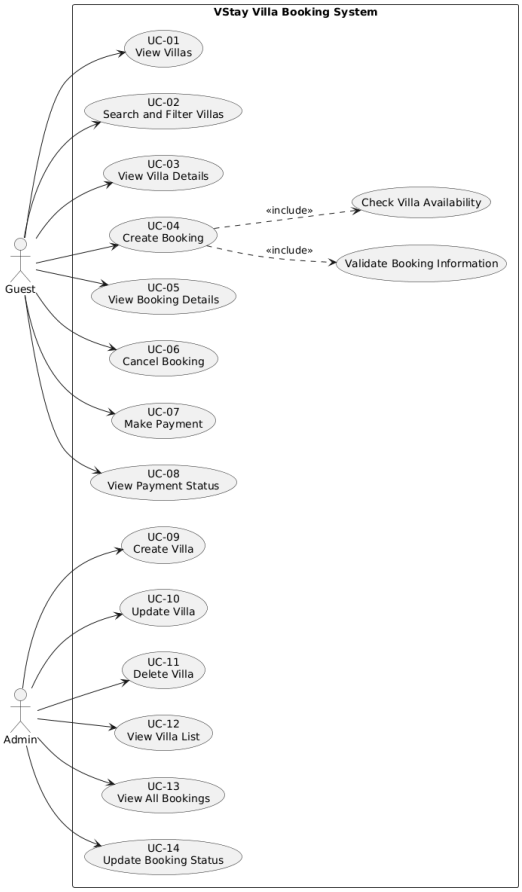

Figure 1: Use Case Diagram of VStay

3.4 Short Explanation

The use case diagram shows two main actors: Guest and Admin. Guests handle villa browsing, booking, cancellation, and payment actions. Admins manage villa records and booking statuses. The Create Booking use case includes validation and availability checking because these are mandatory business rules.

4 Package Diagram

4.1 AI Prompt Used

```
Create a UML package diagram for a Spring Boot villa booking management system.
```

```
The system includes villa browsing, booking management, payment processing
using MomoPay and VNPay, and admin management.
Use a layered architecture with controller, service, payment, repository,
entity, enums, dto, mapper, exception, and config packages.
Include Strategy Pattern and Factory Pattern for payment processing.
```

4.2 UML Code

```
@startuml
skinparam packageStyle rectangle
skinparam shadowing false
skinparam linetype ortho

package "vstay" {

  package "controller" {

    class VillaController
    class BookingController
    class PaymentController

    class AdminVillaController
    class AdminBookingController
  }

  package "service" {

    class VillaService
    class BookingService
    class PaymentService
  }

  package "payment" {
    class PaymentStrategyFactory

    interface PaymentStrategy

    class MomoPaymentStrategy
    class VNPayPaymentStrategy
  }

  package "repository" {

    interface VillaRepository
    interface BookingRepository
    interface PaymentRepository
    interface UserRepository
  }

  package "entity" {
```

```
    class User
    class Villa
    class Booking
    class Payment
  }

  package "enums" {

    enum Role
    enum BookingStatus
    enum PaymentStatus
  }

  package "dto" {

    class VillaResponse
    class BookingRequest
    class BookingResponse

    class PaymentRequest
    class PaymentResponse

    class CreateVillaRequest
    class UpdateVillaRequest
  }

  package "mapper" {

    class VillaMapper
    class BookingMapper
    class PaymentMapper
  }

  package "exception" {

    class NotFoundException
    class ValidationException
    class BookingConflictException
    class AccessDeniedException
  }

  package "config" {

    class SecurityConfig
  }
}

' ========================
' CONTROLLER -> SERVICE
' ========================
```

```
VillaController --> VillaService
BookingController --> BookingService
PaymentController --> PaymentService

AdminVillaController --> VillaService
AdminBookingController --> BookingService

' ========================
' SERVICE -> REPOSITORY
' ========================

VillaService --> VillaRepository
BookingService --> BookingRepository
PaymentService --> PaymentRepository
BookingService --> VillaRepository

' ========================
' SERVICE -> ENTITY
' ========================

VillaService --> Villa
BookingService --> Booking
PaymentService --> Payment

' ========================
' DTO RELATIONSHIPS
' ========================

VillaController --> VillaResponse

BookingController --> BookingRequest
BookingController --> BookingResponse

PaymentController --> PaymentRequest
PaymentController --> PaymentResponse

AdminVillaController --> CreateVillaRequest
AdminVillaController --> UpdateVillaRequest

' ========================
' MAPPER RELATIONSHIPS
' ========================

VillaService --> VillaMapper
BookingService --> BookingMapper
PaymentService --> PaymentMapper

VillaMapper --> Villa
BookingMapper --> Booking
PaymentMapper --> Payment
```

```
' ========================
' STRATEGY PATTERN
' ========================

PaymentService --> PaymentStrategyFactory

PaymentStrategyFactory --> PaymentStrategy

PaymentStrategy <|.. MomoPaymentStrategy
PaymentStrategy <|.. VNPayPaymentStrategy

PaymentService --> PaymentStrategy

' ========================
' EXCEPTION HANDLING
' ========================

VillaService --> ValidationException
BookingService --> BookingConflictException
PaymentService --> ValidationException

' ========================
' SECURITY
' ========================

SecurityConfig --> Role
User --> Role

Booking --> BookingStatus
Payment --> PaymentStatus

@enduml
```

4.3 UML Image

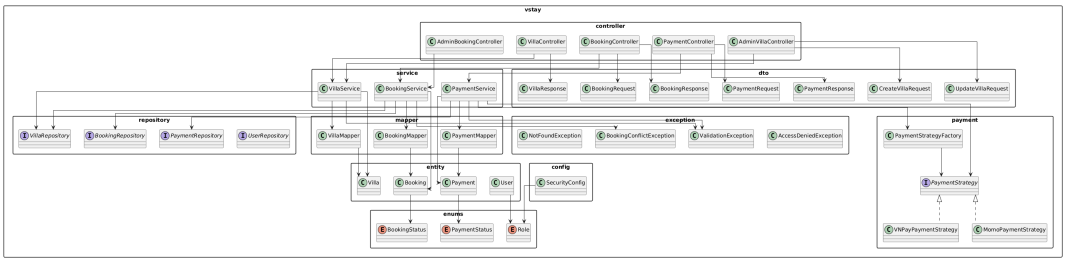

Figure 2: Package Diagram of the VStay Backend

4.4 Short Explanation

The package diagram describes the Spring Boot backend using a layered structure. Controllers receive API requests, services handle business logic, repositories access data, entities and enums represent the domain model, DTOs and mappers control API data

transfer, exceptions centralize error handling, and config stores security configuration. The payment package applies Strategy and Factory patterns for provider-based payment processing.

5 Class Diagram

5.1 AI Prompt Used

```
Create a simple UML class diagram for a villa booking management system.

The system includes guests and admins, villas, bookings, payments, booking
status, and payment status.
Only include the main domain classes, their basic attributes, simple methods,
enum values, and relationships.
Do not include technical implementation layers.
```

5.2 UML Code

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false

class User {
  - id: Long
  - fullName: String
  - email: String
  - phone: String
  - role: Role
  + isAdmin(): boolean
  + isGuest(): boolean
}

class Villa {
  - id: Long
  - name: String
  - location: String
  - description: String
  - pricePerNight: BigDecimal
  - capacity: int
  - active: boolean
  + updateInfo(): void
  + deactivate(): void
}

class Booking {
  - id: Long
  - checkInDate: LocalDate
  - checkOutDate: LocalDate
  - numberOfGuests: int
```

```
  - contactName: String
  - contactPhone: String
  - status: BookingStatus
  + confirm(): void
  + cancel(): void
  + complete(): void
  + isActive(): boolean
}

class Payment {
  - id: Long
  - provider: String
  - amount: BigDecimal
  - status: PaymentStatus
  - transactionCode: String
  - paidAt: LocalDateTime
  + markSuccessful(): void
  + markFailed(): void
}

enum Role {
  GUEST
  ADMIN
}

enum BookingStatus {
  PENDING
  CONFIRMED
  CANCELLED
  COMPLETED
}

enum PaymentStatus {
  PENDING
  SUCCESSFUL
  FAILED
}

User "1" --> "0..*" Booking : creates
Villa "1" --> "0..*" Booking : is booked in
Booking "1" --> "0..1" Payment : has

User --> Role
Booking --> BookingStatus
Payment --> PaymentStatus

@enduml
```

5.3 UML Image

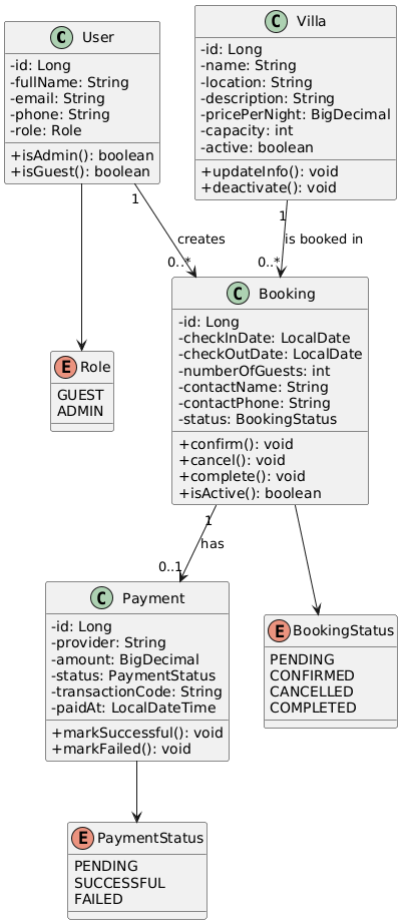

Figure 3: Core Domain Class Diagram

5.4 Short Explanation

The class diagram focuses on four core domain classes: User, Villa, Booking, and Payment. User represents both guests and admins through the Role enum. Villa stores villa information and active status. Booking connects a user to a villa for a date range and tracks booking status. Payment belongs to one booking and stores the simplified payment result.

6 Sequence Diagrams

The sequence diagrams are separated by workflow to keep each behavior readable and aligned with the use cases.

6.1 AI Prompt Used

```
Create simple UML sequence diagrams for a villa booking management system.

Actors:
- Guest
```

```
- Admin

Separate the sequence diagrams by action:
- Guest browses, searches, and views villas
- Guest creates a booking
- Guest cancels a booking
- Guest makes a payment and views payment status
- Admin manages villas
- Admin views bookings and updates booking status

Keep each diagram high-level and business-focused.
Do not include technical implementation layers.
```

6.2 Browse and View Villas

```
@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor Guest
participant "VStay System" as System
database "Stored Data" as Data

Guest -> System: Request villa list or search result
System -> Data: Load matching villas
Data --> System: Villa list
System --> Guest: Show villas

Guest -> System: Select villa
System -> Data: Load villa details
Data --> System: Villa details
System --> Guest: Show villa details

@enduml
```

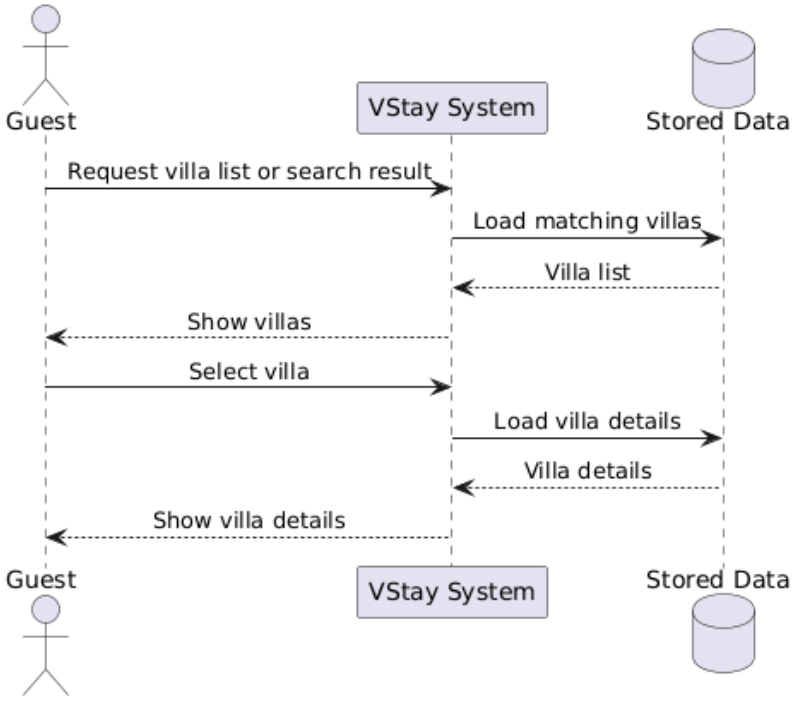

Figure 4: Sequence Diagram for Browsing and Viewing Villas

The guest requests a villa list or search result. The system loads matching villas and returns the list or selected villa details.

6.3 Create Booking

```
@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor Guest
participant "VStay System" as System
database "Stored Data" as Data

Guest -> System: Submit booking information
System -> System: Validate dates, guest count, and contact information
System -> Data: Check villa availability
Data --> System: Availability result

alt Villa is not available
  System --> Guest: Show booking conflict message
else Villa is available
  System -> Data: Save booking with PENDING status
  Data --> System: Saved booking
  System --> Guest: Show booking details
end

@enduml
```

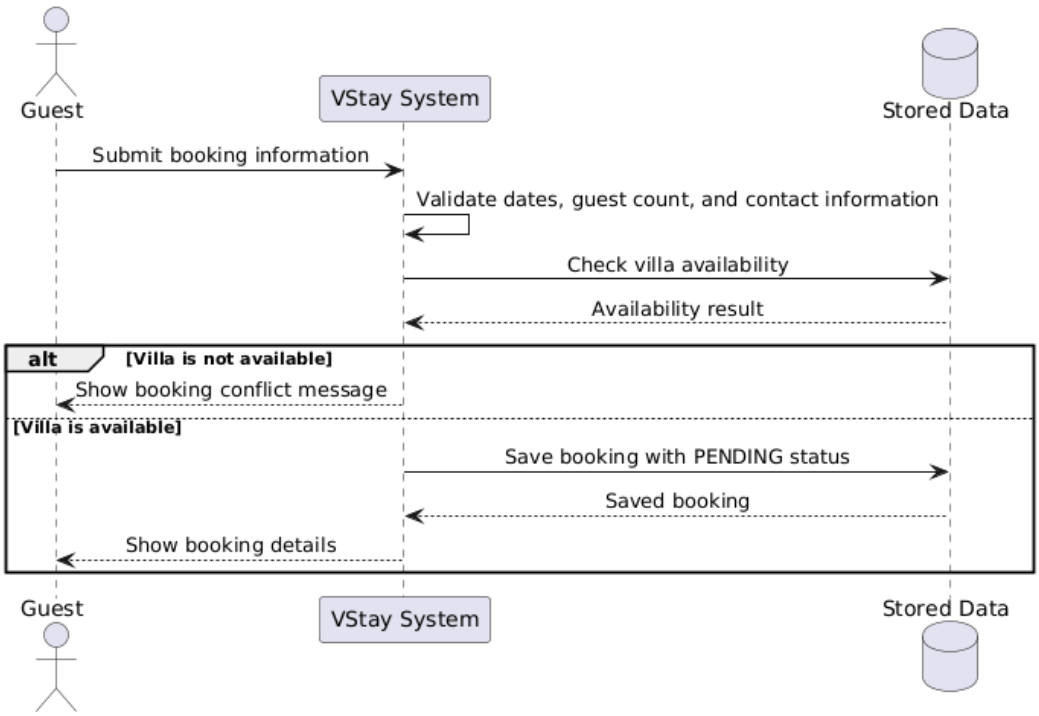

Figure 5: Sequence Diagram for Creating a Booking

The guest submits booking information. The system validates dates, guest count, contact information, and villa availability. If valid, the booking is saved with PENDING status.

6.4 Cancel Booking

```
@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor Guest
participant "VStay System" as System
database "Stored Data" as Data

Guest -> System: Request booking cancellation
System -> Data: Load booking
Data --> System: Booking details
System -> System: Check cancellation eligibility

alt Booking can be cancelled
  System -> Data: Update booking status to CANCELLED
  Data --> System: Updated booking
  System --> Guest: Show cancelled booking
else Booking cannot be cancelled
  System --> Guest: Show validation message
end

@enduml
```

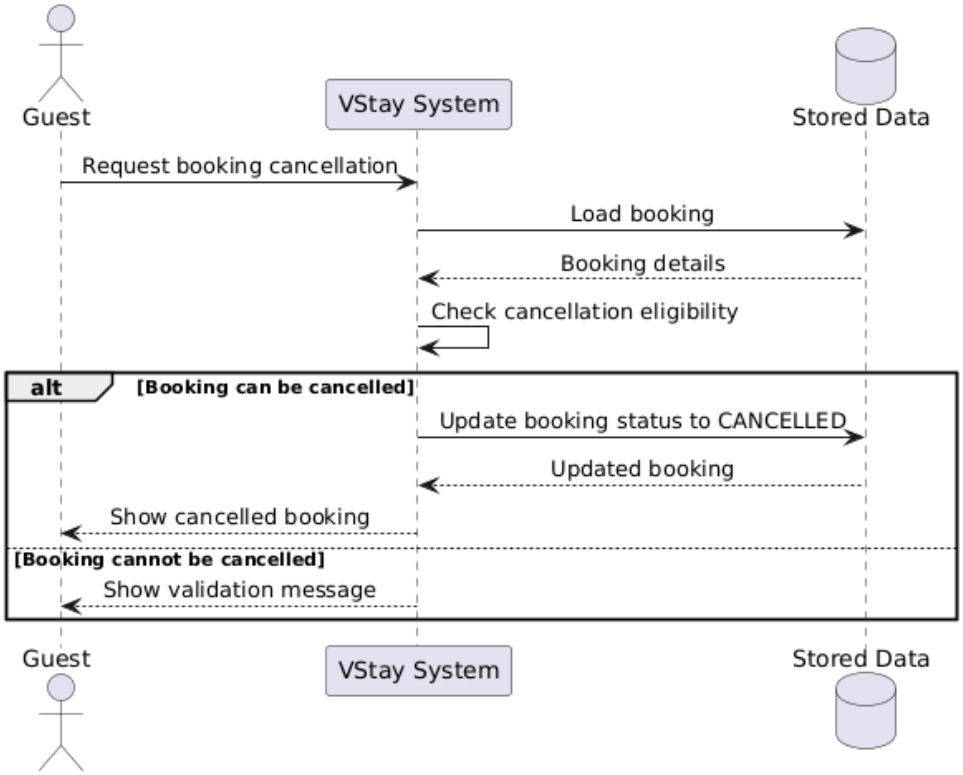

Figure 6: Sequence Diagram for Cancelling a Booking

The guest requests booking cancellation. The system loads the booking, checks ownership and cancellation eligibility, then updates the status to CANCELLED if the request is valid.

6.5 Make Payment and View Payment Status

```
@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor Guest
participant "VStay System" as System
database "Stored Data" as Data

Guest -> System: Submit payment information
System -> Data: Load booking
Data --> System: Booking details
System -> System: Process simplified payment

alt Payment is successful
  System -> Data: Save payment with SUCCESSFUL status
  System -> Data: Update booking status to CONFIRMED
  Data --> System: Updated payment and booking
  System --> Guest: Show successful payment status
else Payment fails or is pending
  System -> Data: Save payment with FAILED or PENDING status
  Data --> System: Saved payment
```

```
  System --> Guest: Show payment status
end

Guest -> System: Request payment status
System -> Data: Load payment by booking
Data --> System: Payment status
System --> Guest: Show payment status

@enduml
```

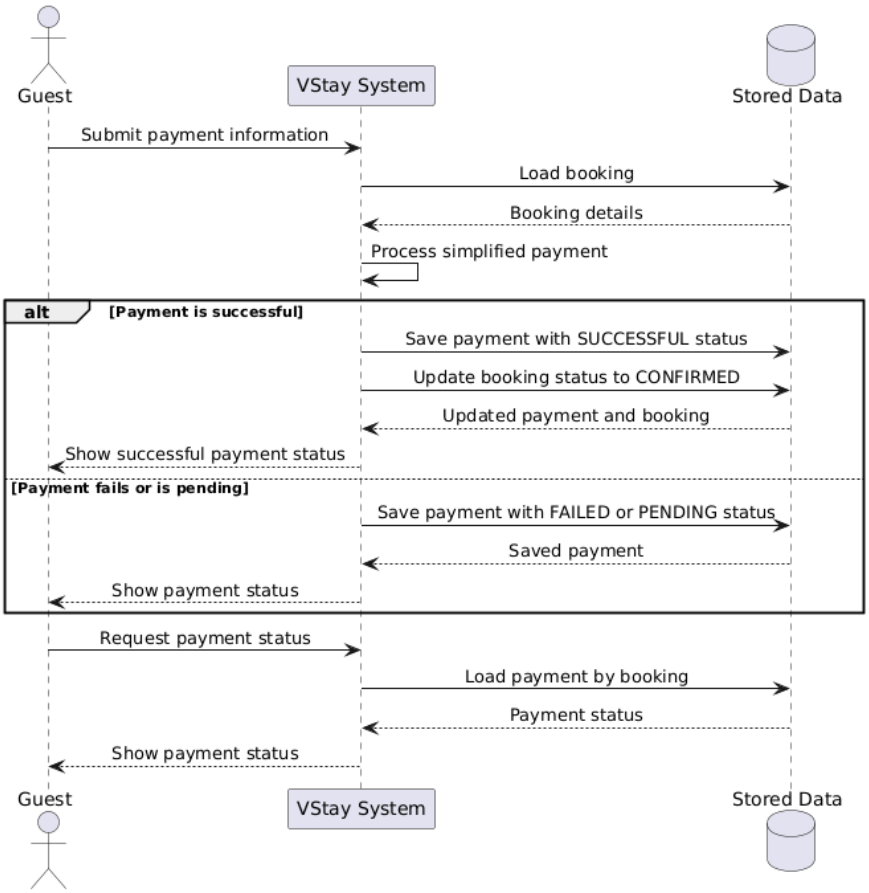

Figure 7: Sequence Diagram for Payment and Payment Status

The guest submits payment for a booking. The system selects a payment strategy, records the payment result, and confirms the booking if the payment succeeds. The guest can also request the payment status later.

6.6 Admin Manage Villas

```
@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor Admin
participant "VStay System" as System
```

```
database "Stored Data" as Data

Admin -> System: Create or update villa information
System -> System: Validate villa information
System -> Data: Save villa information
Data --> System: Saved villa
System --> Admin: Show updated villa list

Admin -> System: Delete villa
System -> Data: Check active bookings for villa
Data --> System: Active booking result

alt Villa has active booking
  System --> Admin: Reject villa deletion
else Villa has no active booking
  System -> Data: Delete villa
  Data --> System: Villa deleted
  System --> Admin: Show updated villa list
end

@enduml
```

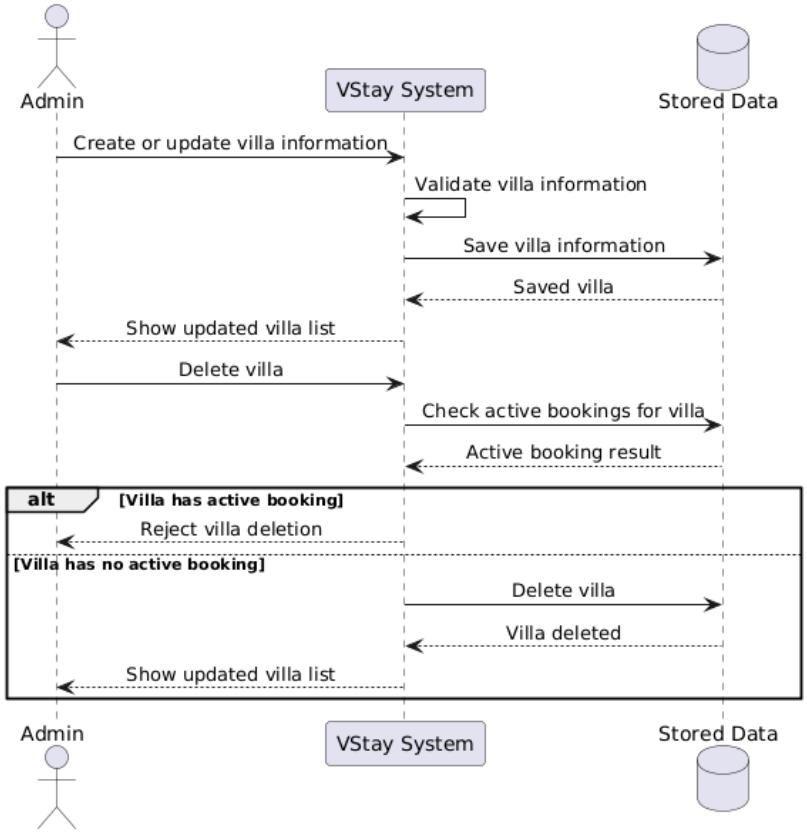

Figure 8: Sequence Diagram for Admin Villa Management

The admin creates or updates villa information after validation. When deleting a villa, the system checks for active bookings and rejects deletion if active bookings still exist.

6.7 Admin Manage Bookings

```
@startuml
skinparam shadowing false
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor Admin
participant "VStay System" as System
database "Stored Data" as Data

Admin -> System: Request all bookings
System -> Data: Load bookings
Data --> System: Booking list
System --> Admin: Show booking list

Admin -> System: Update booking status
System -> System: Validate status change
System -> Data: Save updated booking status
Data --> System: Updated booking
System --> Admin: Show updated booking

@enduml
```

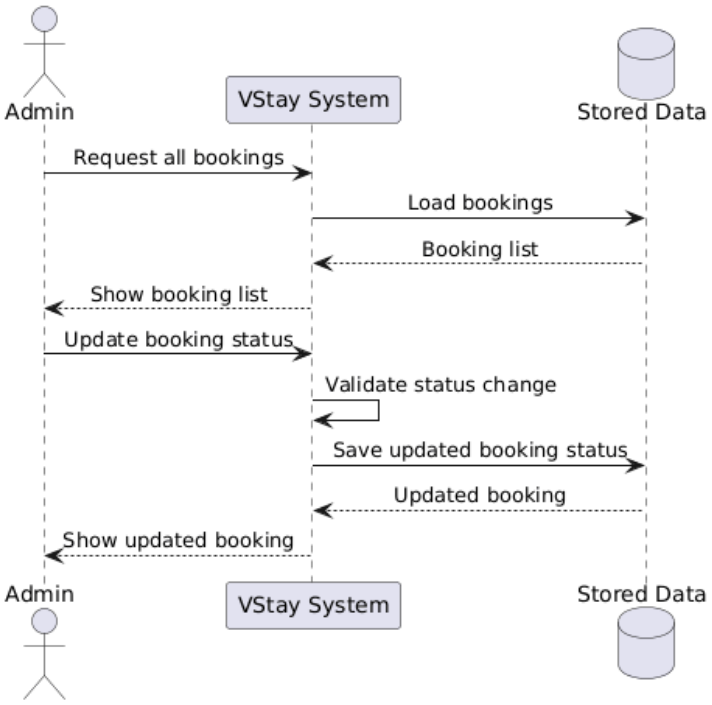

Figure 9: Sequence Diagram for Admin Booking Management

The admin views all bookings and updates booking status. The system validates status transitions to keep booking state consistent.

7 Design Patterns

Pattern	Applied In	Purpose
Layered Architecture	Controller, service, repository, and entity packages	Separates API handling, business logic, data access, and domain representation.
Repository Pattern	Repository interfaces	Hides database query details and allows services to work through clean repository interfaces.
DTO & Mapper Pattern	DTO and mapper packages	Prevents direct exposure of entities through APIs and centralizes request/response conversion.
Strategy Pattern	Payment strategy classes	Separates payment behavior by provider and makes it easier to add new providers.
Factory Pattern	Payment strategy factory	Selects the correct payment strategy by provider and keeps the payment service decoupled from concrete classes.

7.1 Design Pattern UML Note

PlantUML files for the design patterns are available in `docs/uml`. The relevant files are:

- `design-pattern-layered-architecture.puml`
- `design-pattern-repository.puml`
- `design-pattern-dto-mapper.puml`
- `design-pattern-strategy.puml`
- `design-pattern-factory.puml`

8 Code Implementation

The backend prototype is implemented with Java Spring Boot, Maven, Spring Data JPA, Spring Security, Swagger/OpenAPI, and an H2 demo database. The main source code is located in `backend/src/main/java/com/kna/vstay`.

- GitHub repository: https://github.com/kenji-cmyk/mini-booking-villa
- Public APIs: `/api/villas`, `/api/bookings`, `/api/payments`
- Admin APIs: `/api/admin/villas`, `/api/admin/bookings`
- Local Swagger UI: `http://localhost:8080/swagger-ui.html`

8.1 Implementation Status

The implementation covers the main use cases from UC-01 to UC-14. Important rules are implemented, including overlapping booking prevention, guest count validation, guest ownership checks for booking and payment actions, payment statuses PENDING, SUCCESSFUL, and FAILED, and valid booking status transitions.

9 Conclusion

VStay satisfies the assignment scope with requirements, use cases, UML diagrams, design patterns, and a backend implementation prototype. The system uses layered architecture, repository, DTO/mapper, strategy, and factory patterns to keep the design clear, maintainable, and suitable for academic demonstration. The main Guest and Admin workflows are described through sequence diagrams and implemented in the backend prototype.

10 References

[1] Object Management Group. *OMG Unified Modeling Language Specification.* https://www.omg.org/spec/UML/

[2] PlantUML. *Use Case Diagram Syntax and Features.* https://plantuml.com/use-case-diagram

[3] PlantUML. *Class Diagram Syntax and Features.* https://plantuml.com/class-diagram

[4] PlantUML. *Sequence Diagram Syntax and Features.* https://plantuml.com/sequence-diagram

[5] PlantUML. *Deployment and Package Diagram Syntax.* https://plantuml.com/deployment-diagram

[6] Visual Paradigm. *UML Diagram Tutorials and Examples.* https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/

[7] Martin Fowler. *Patterns of Enterprise Application Architecture.* Addison-Wesley, 2002.

[8] Refactoring.Guru. *Design Patterns.* https://refactoring.guru/design-patterns

[9] Spring. *Spring Boot Reference Documentation.* https://docs.spring.io/spring-boot/reference/

[10] Spring. *Spring Data JPA Reference Documentation.* https://docs.spring.io/spring-data/jpa/reference/